

DISTRIBUTED SYSTEMS

HTTP over RPC

Scalable Proxy Architecture with MapReduce Analytics & Distributed Caching

Tech Stack: Python, XML-RPC, Docker Compose



Project Overview

The Challenge

Build a transparent **HTTP Proxy** that distributes client requests across multiple worker nodes to ensure high availability and performance.

Key Objectives

- ✓ **Scalability:** Handle concurrent requests.
- ✓ **Consistency:** Unified data view (Cache).
- ✓ **Analytics:** Insight into user traffic.

```
Request 1 → Worker 1 → Processed  
Request 2 → Worker 2 → Processed in parallel  
Request 3 → Worker 3 → Processed in parallel  
→ System handles multiple requests concurrently
```

```
100 incoming requests → Proxy  
|  
Load Balancer (Round-robin)  
|  
Worker 1 (~33 req)  Worker 2 (~33 req)  Worker 3 (~33 req)
```

```
user@group6:~$ curl -x localhost:8080 http://example.com  
  
[Proxy] Receiving Request...  
[Round-Robin] Forwarding to Worker-2  
[DFS] Cache MISS → Fetching from Internet  
[Success] Returning 200 OK  
[Log] Written to access.log
```

If Worker 1 fails:

```
Worker 1 : ✗ Dead  
Worker 2 : ✓ Running  
Worker 3 : ✓ Running
```

→ System continues operating (2/3 workers alive)

Request Processing Flow

Request Reception

- 01 Proxy receives HTTP request. Checks **Shared Cache** (Docker Volume) first.

Load Balancing (Round Robin)

- 02 If Cache Miss, forward to Worker using: `worker_id = index % numWorkers`.

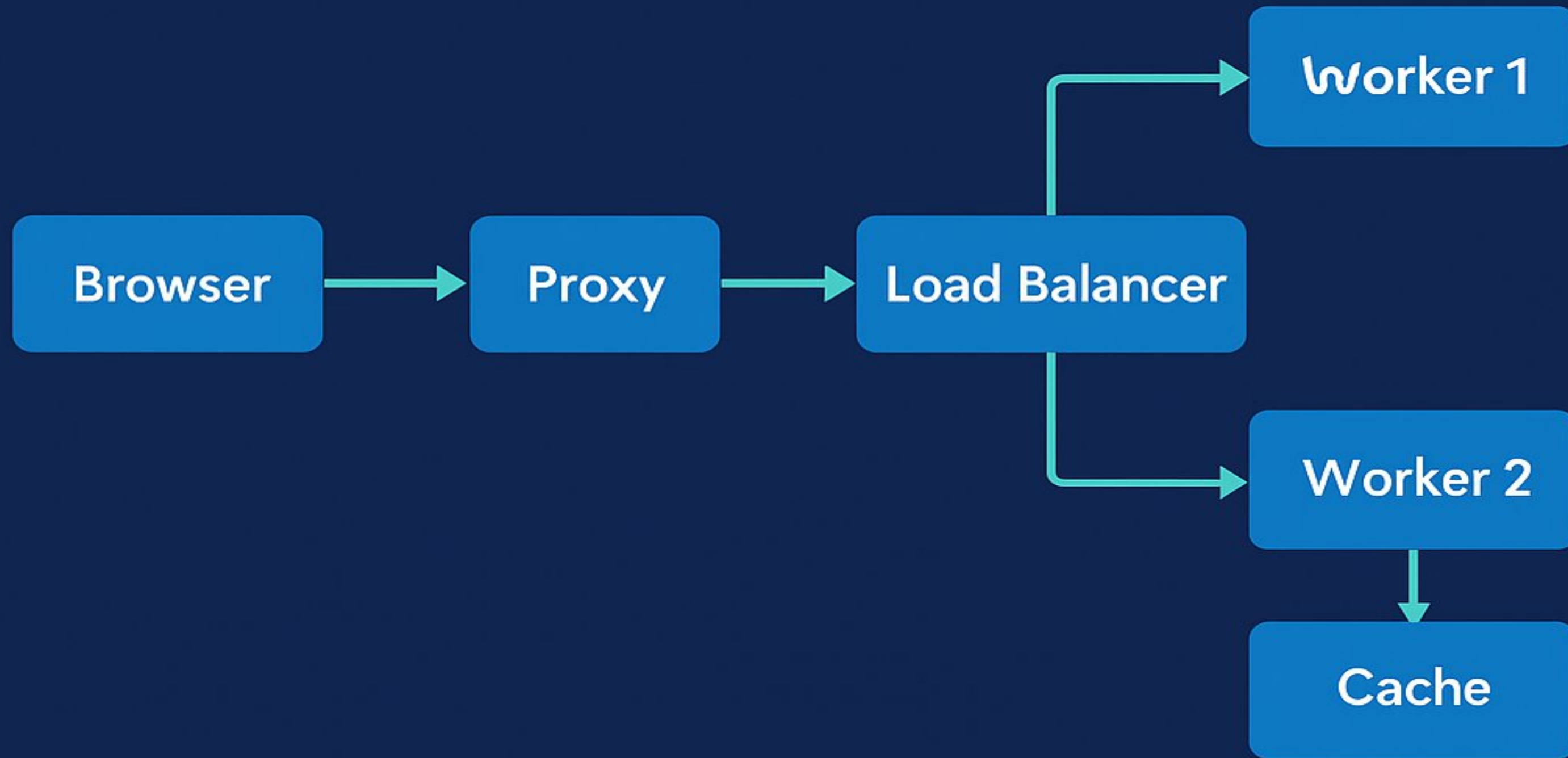
Worker Execution

- 03 Worker fetches data from Internet -> Writes to `/app/cache_data` -> Returns result.

Logging

- 04 Request details (Latency, Worker ID, Status) are appended to `access.log`.

SYSTEM ARCHITECTURE



Core Implementation Details

Load Balancing

Implemented **Round Robin** algorithm. Ensures requests are evenly distributed across 3 active workers, preventing any single node from overloading.

Distributed FS (DFS)

Achieved via **Docker Volumes**. Proxy and all Workers mount the same `/cache` directory. Data written by Worker 1 is instantly available to Worker 3.

```
# rpc_proxy_server.py (Snippet)

def get_next_worker(self):
    with self.lock:
        worker = self.workers[self.index]
        # Round Robin Logic
        self.index = (self.index + 1) % len(self.workers)
    return worker
```

```
rpc-worker-3 | 172.18.0.5 - - [09/Dec/2025 14:46:16] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-2 exited with code 137
rpc-worker-1 | 172.18.0.5 - - [09/Dec/2025 14:46:26] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-3 | 172.18.0.5 - - [09/Dec/2025 14:46:30] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-1 | 172.18.0.5 - - [09/Dec/2025 14:46:40] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-3 | 172.18.0.5 - - [09/Dec/2025 14:46:44] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-1 | 172.18.0.5 - - [09/Dec/2025 14:46:54] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-3 | 172.18.0.5 - - [09/Dec/2025 14:46:58] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-1 | 172.18.0.5 - - [09/Dec/2025 14:47:06] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-1 | 172.18.0.5 - - [09/Dec/2025 14:47:08] "POST /RPC2 HTTP/1.1" 200 -
rpc-worker-3 | 172.18.0.5 - - [09/Dec/2025 14:47:12] "POST /RPC2 HTTP/1.1" 200 -
```


Analytics: MapReduce Pipeline

1. Access Log Format

Every request generates a structured log entry containing Timestamp, Client IP, URL, Worker ID, and Latency.

```
2023-10-27T10:00:01 | 192.168.1.5 | GET | google.com | 200 | worker-1 | 0.45s
2023-10-27T10:00:05 | 192.168.1.5 | GET | youtube.com | 200 | worker-2 | 0.82s
```

2. The Pipeline

Map: Parse logs & emit `(Domain, 1)` pairs.

Reduce: Sum counts to find Top Visited Sites.

```
PS D:\Project3_HTTP_RPC 2\Project3_HTTP_RPC> docker-compose --profile tools run analytics
[MapReduce] Log file: /app/logs/access.log
=====
[MapReduce] Starting Log Analysis
=====
[MapReduce] Processing 30 log entries...

[MAP] Parsing log entries...
[MAP] Generated 210 key-value pairs

[SHUFFLE] Grouping by keys...
[SHUFFLE] Created 18 groups

[REDUCE] Aggregating values...

=====
ANALYTICS REPORT
=====

📊 TOP 10 DOMAINS (Most Accessed):
-----
1. example.com          20 ██████████
2. httpbin.org          10 ████████

🗄️ CACHE STATISTICS:
-----
Cache Hits:    13 (43.3%)
Cache Misses:  17 (56.7%)
Total:         30
[ ●●●●●●●●●●●●●●●● ]

⚙️ WORKER DISTRIBUTION (Load Balancing):
-----
worker-1      12 ██████████
worker-3       5 ████████
worker-2       5 ████████
local         3 ████████

⌚ RESPONSE TIME:
-----
Average Response Time: 618.74 ms
Total Requests:        30

📋 HTTP STATUS CODES:
-----
✅ 200: 25
❌ 503: 5

=====
[MapReduce] Report exported to: logs/analytics_report.csv
```

Deployment & Testing

Docker Orchestration

We use `docker-compose` to define the infrastructure code.

- **Service 1:** Proxy Server (Port 8080)
- **Service 2-4:** RPC Workers (Scaled x3)
- **Volume:** Shared `/cache_data`
- **Network:** Internal Bridge Network

```
$ docker-compose up --build
[+] Running 4/4
Container rpc-worker-1 Started
Container rpc-worker-2 Started...
```

Performance Tests

Using `test_proxy.py` to simulate stress loads.

```
PS D:\Project3_HTTP_RPC_2\Project3_HTTP_RPC> docker-compose exec proxy python test_proxy.py
time="2025-12-09T21:44:37+07:00" level=warning msg="D:\\Project3_HTTP_RPC_2\\Project3_HTTP_RPC\\
red, please remove it to avoid potential confusion"
=====
HTTP RPC PROXY - TEST SUITE
=====
Proxy: http://localhost:8080

=====
TEST 1: CACHE CONSISTENCY (DFS)
=====
All workers share the same cache directory.
First request = FRESH, subsequent = CACHED

--- Request 1 ---

[Test] Fetching: http://example.com
[Test] Status: 200
[Test] Time: 0.638s
[Test] X-Cached: False
[Test] X-Worker: worker-1
[Test] Content-Length: 513 bytes

--- Request 2 ---

[Test] Fetching: http://example.com
[Test] Status: 200
[Test] Time: 0.012s
[Test] X-Cached: True
[Test] X-Worker: worker-2
[Test] Content-Length: 513 bytes

--- Request 3 ---

[Test] Fetching: http://example.com
[Test] Status: 200
[Test] Time: 0.013s
[Test] X-Cached: True
[Test] X-Worker: worker-3
[Test] Content-Length: 513 bytes
```